



Estruturas de Dados

em Linguagem C

AULA 13

Profª. Drª. Tatiele
Bolson Moro

MATRIZES

Vetores Bidimensionais em C

Ciência da Computação

Programação em C



O QUE VAMOS APRENDER

01 O que é uma Matriz?

02 Declaração e Sintaxe

03 Índices e Propriedades

04 Acesso e Manipulação

05 Laços Aninhados (for-for)

06 printf e scanf em Matrizes

07 Exemplos Práticos (1–4)



O QUE É UMA MATRIZ?

Conceito

Uma matriz é uma **estrutura de dados bidimensional** que organiza elementos em **linhas e colunas**.

Cada elemento é identificado por um **par de índices**: um para a linha e outro para a coluna.

Pense como uma planilha ou tabela de dados.

Representação Visual

	0	1	2	3
0	5	3	8	2
1	1	7	4	9
2	6	0	3	1

A[1][2] = 4 ← elemento na linha 1, coluna 2



VETOR vs MATRIZ



VETOR (1D)

```
int vet[10];
```

5	6	-7	2	0	-98	56	23	1	1
0	1	2	3	4	5	6	7	8	9

→ Um único índice: `vet[i]`

→ Armazena dados em uma linha

→ Acesso simples e direto

VS



MATRIZ (2D)

```
int mat[5][5];
```

6	2	-9	8	0
3	6	55	1	2
8	-5	2	8	1
4	7	32	7	1
2	3	9	8	0

→ Dois índices: `mat[i][j]`

→ Organiza dados em linhas e colunas

→ Elemento `mat[3][3]` destacado

</> DECLARAÇÃO DE MATRIZES

Sintaxe:

```
tipo  nome [ qtde_linhas ][ qtde_colunas ];
```

tipo

Tipo dos elementos: int, float, char...

nome

Identificador da variável matriz

qtde_linhas

Número de linhas da matriz

qtde_colunas

Número de colunas da matriz

Exemplo: `int mat[5][5];` `float notas[3][4];` `char nome[10][50];`

ÍNDICES E PROPRIEDADES

⚡ Os índices SEMPRE iniciam em ZERO

Linhas

$[0, L-1]$

5 linhas $\rightarrow$ índices 0 a 4



Colunas

$[0, C-1]$

10 colunas $\rightarrow$ índices 0 a 9



Acesso

`mat[L][C]`

Linha L, Coluna C

Exemplo: `int A[5][10];` $\rightarrow$ linhas: `A[0]...A[4]` | colunas: `A[?][0]...A[?][9]`



REPRESENTAÇÃO DE MATRIZES

Notação Matemática

Matriz A[2,3]:

$$a_{11}=4$$

$$a_{12}=16$$

$$a_{13}=25$$

$$a_{21}=81$$

$$a_{22}=100$$

$$a_{23}=9$$

Leitura: $a_{\text{linha}, \text{coluna}}$

A notação matemática usa 1-based.

Em C, os índices começam em 0!

Notação em C (0-based)

```
int A[2][3];
```

$$A[0][0]=4$$

$$A[0][1]=16$$

$$A[0][2]=25$$

$$A[1][0]=81$$

$$A[1][1]=100$$

$$A[1][2]=9$$

Índice linha: 0 ou 1

Índice coluna: 0, 1 ou 2

Total: $2 \times 3 = 6$ elementos



MANIPULAÇÃO: ACESSO AOS ELEMENTOS

```
float A[2][2];  
A[0][0] = 9.0;  
A[0][1] = 8.0;  
A[1][0] = 7.0;  
A[1][1] = 7.0;
```

```
float quant;  
quant = A[0][0] + A[0][1] +  
        A[1][0] + A[1][1];  
printf("%f", quant);
```

► 31.000000 ← Soma dos elementos

Visualização da Matriz

[0][0] 9.0	[0][1] 8.0
[1][0] 7.0	[1][1] 7.0

```
float media;  
media = quant / 4;  
printf("%f", media);
```

► 7.750000 ← Média dos elementos

1 2 3 LAÇOS ANINHADOS

Para matrizes usamos um for dentro de outro for — chamamos de laços aninhados.

```
int i, j;
int A[3][4];
for(i = 0; i < 3; i++) // percorre as LINHAS
{
    for(j = 0; j < 4; j++) // percorre as COLUNAS
    {
        printf("A[%d][%d] = %d\n", i, j, A[i][j]);
    }
}
```

Laço externo
controla as LINHAS
(i = 0 até 2)

Laço interno
controla as COLUNAS
(j = 0 até 3)

Acessa cada
elemento com
dois índices [i][j]

Total de iterações: 3 linhas × 4 colunas = 12 acessos



LEITURA E ESCRITA DE MATRIZES



SAÍDA: printf

```
for(i=0; i<3; i++) {  
    for(j=0; j<4; j++) {  
        printf("A[%d][%d]=%d\n",  
            i, j, A[i][j]);  
    }  
}
```

Imprime cada elemento com sua posição [linha][coluna]



ENTRADA: scanf

```
for(i=0; i<3; i++) {  
    for(j=0; j<4; j++) {  
        printf("A[%d][%d]:", i, j);  
        scanf("%d", &A[i][j]);  
    }  
}
```

Lê elemento por elemento.
Nota: & para obter endereço!



A combinação printf + scanf dentro dos laços aninhados permite controle total da entrada e saída!

EXEMPLO PRÁTICO: Leitura de Matriz 3×4

01

 **Objetivo:** ler os elementos de uma matriz 3×4 via teclado e exibi-los na tela

```
#include <stdio.h>
#include <conio.h>
int main() {
    int i, j, mat[3][4];
    for(i = 0; i < 3; i++) {
        for(j = 0; j < 4; j++) {
            printf("Digite o elemento [%d][%d]: ", i, j);
            scanf("%d", &mat[i][j]);
        }
    }
    for(i = 0; i < 3; i++) {
        for(j = 0; j < 4; j++) {
            printf("%d \t", mat[i][j]);
        }
        printf("\n");
    }
    getch();
}
```

Matriz 3×4

3 linhas × 4 colunas
= 12 elementos lidos

Dois laços for

Externo: linhas (i)
Interno: colunas (j)

Saída em Tabela

printf("\n") ao final
de cada linha gera
formato tabular

EXEMPLO PRÁTICO: Média de Matriz 3×4

02

 Objetivo: preencher $\text{mat}[i][j] = i*j$ e calcular a média de todos os elementos

```
#include <stdio.h>
#include <conio.h>
int main() {
    int i, j;
    float mat[3][4], soma=0, media=0;
    for(i=0; i<3; i++) {
        for(j=0; j<4; j++) {
            mat[i][j] = i * j;    // atribui i*j
        }
    }
    for(i=0; i<3; i++) {
        for(j=0; j<4; j++) {
            soma += mat[i][j];    // acumula soma
        }
    }
    media = soma / (i * j);
    printf("media: %f", media);
    getch();
}
```

Matriz resultante $\text{mat}[i][j]=i*j$

0	0	0	0
0	1	2	3
0	2	4	6

Resultado

$\text{soma} = 0+1+2+3+2+4+6 = 18$
 $i=3, j=4 \rightarrow \text{media} = 18/12$
 $= 1.500000$

EXEMPLO PRÁTICO: Diagonal Principal (4×4)

03

🎯 **Objetivo:** preencher `mat[i][j]=i*j`, imprimir toda a matriz e depois apenas a diagonal principal (`i==j`)

```
#include <stdio.h>
#include <conio.h>
int main() {
    int i, j;
    float mat[4][4];
    for(i=0; i<4; i++)
        for(j=0; j<4; j++)
            mat[i][j] = i * j;
    for(i=0; i<4; i++) { // Imprime toda a matriz
        for(j=0; j<4; j++)
            printf("%.2f ", mat[i][j]);
        printf("\n");
    }
    for(i=0; i<4; i++) // Diagonal principal:
        somente quando i==j
        for(j=0; j<4; j++)
            if(i == j)
                printf("%.2f ", mat[i][j]);
    getch();
}
```

Chave: `if(i == j)` → apenas diagonal principal!

Matriz 4×4

0.0	0.0	0.0	0.0
0.0	1.0	2.0	3.0
0.0	2.0	4.0	6.0
0.0	3.0	6.0	9.0

Diagonal: 0.0 1.0 4.0 9.0

EXEMPLO PRÁTICO: Acessar Coluna Específica (5×5)

04

🎯 Objetivo: imprimir toda $\text{mat}[5][5]$ ($\text{mat}[i][j]=i*j$), depois a 1ª coluna $[i][0]$ e a 4ª coluna $[i][3]$

```
#include <stdio.h>
#include <conio.h>
int main() {
    int i, j;
    float mat[5][5];
    for(i=0; i<5; i++)
        for(j=0; j<5; j++) {
            mat[i][j] = i * j;
            printf("%.2f ", mat[i][j]);
        } printf("\n");
    // Coluna 0
    for(i=0; i<5; i++)
        printf("%.2f ", mat[i][0]);
    // Coluna 3
    for(i=0; i<5; i++)
        printf("%.2f ", mat[i][3]);
    getch();
}
```

Matriz 5×5 ($\text{mat}[i][j] = i*j$)

0	0	0	0	0
0	1	2	3	4
0	2	4	6	8
0	3	6	9	12
0	4	8	12	16

Coluna 0: 0,0,0,0,0

Coluna 3: 0,3,6,9,12

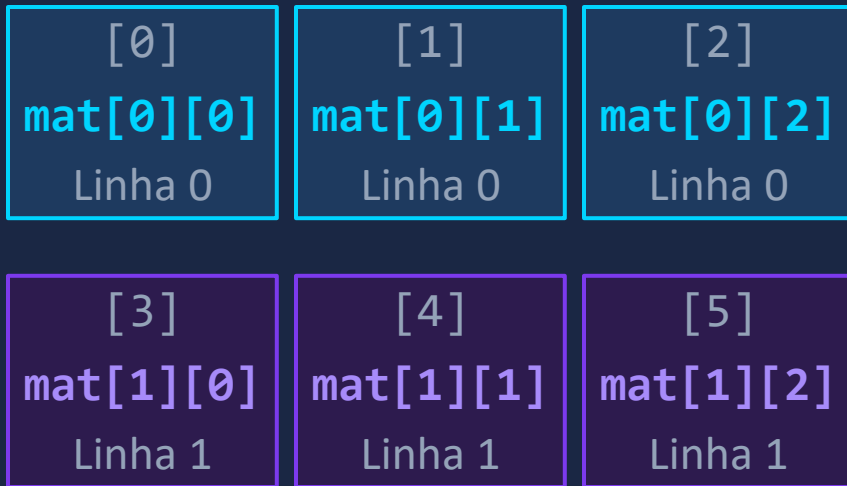


MATRIZES NA MEMÓRIA

Como a memória funciona?

- Em C, a matriz é armazenada em memória de forma contígua (row-major order).
- Os elementos da linha 0 ficam primeiro, seguidos pelos da linha 1, e assim por diante.
- O compilador converte `mat[i][j]` em: $\text{endereço_base} + (i \times \text{colunas} + j) \times \text{tamanho_tipo}$.
- Por isso os índices precisam ser declarados corretamente!

Layout: `int mat[2][3]`



← Contíguo na RAM →



BOAS PRÁTICAS COM MATRIZES



Inicialize a matriz

Sempre inicialize os valores antes de usar, especialmente para somas e médias (soma=0).



Use constantes para dimensões

#define LINHAS 5 → facilita manutenção do código.



Confira os limites dos índices

Acesso fora dos limites (out-of-bounds) causa comportamento indefinido em C.



Cuidado com i e j trocados

mat[j][i] ao invés de mat[i][j] é um erro comum que causa bugs silenciosos.



Use & no scanf

scanf("%d", &mat[i][j]) — não esquecer o endereço-de (&)!



Laços com limites corretos

i < LINHAS e j < COLUNAS, não i <= LINHAS (estouro de memória!).



ONDE MATRIZES SÃO USADAS?



Processamento de Imagens

Pixels de uma imagem são organizados em matrizes de intensidade de cor (R, G, B).



Jogos e Simulações

Tabuleiros de xadrez, mapas de jogos (ex: Minecraft) usam matrizes 2D/3D.



Machine Learning

Redes neurais operam sobre matrizes de pesos e ativações (álgebra linear).



Álgebra Linear

Operações como multiplicação de matrizes são fundamentais em engenharia.



Redes e Grafos

Matrizes de adjacência representam conexões entre nós em redes.



Bioinformática

Alinhamento de sequências de DNA usa matrizes de pontuação (score matrices).



FIXAÇÃO DE CONTEÚDO

1. Qual é a declaração correta de uma matriz 4 linhas × 6 colunas de inteiros?

A) `int mat[6][4];`

B) `int mat[4][6];` ✓

C) `int mat[24];`

D) `matrix mat(4,6);`

2. O elemento `mat[2][3]` está em qual posição?

A) Linha 3, Coluna 4

B) Linha 2, Coluna 3 ✓

C) Posição 23

D) Linha 3, Coluna 2

3. Para uma matriz `int A[5][8]`, qual é o maior índice de coluna válido?

A) 8

B) 5

C) 7 ✓

D) 4



RESUMO: O QUE APRENDEMOS

01

Matriz é uma estrutura 2D com linhas e colunas em linguagem C

03

Índices começam em 0 e vão até tamanho-1

05

Laços aninhados for-for percorrem toda a matriz

07

Diagonal principal: condição especial `if(i==j)`

02

Declaração: `tipo nome[linhas][colunas];`

04

Acesso a elementos: `mat[i][j]`

06

`printf` e `scanf` com duplo for para I/O completo

08

Matrizes têm aplicações em imagens, IA, jogos e mais!

MATRIZES EM C

Fim da Aula 13

Prof^a. Dr^a. Tatiele Bolson Moro — Ciência da Computação



Pratique! A melhor forma de aprender matrizes é escrever e executar código.